

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA0506 — Database Management Systems

Assignment: Real-Time Airline Reservation System (Design-Based)

Model Solution — Complete Set of 13 Deliverables

Entities: Passenger, Flight, Aircraft, Seat, Booking, Payment

Note for the student: This is a complete worked solution covering all 13 required deliverables — problem statement through references and a one-page write-up. Insert your own name(s), register number(s), team member contributions, and screenshots of actual query execution before submission — run the SQL yourself in MySQL/PostgreSQL/Oracle and do not submit this unchanged.

1. Problem Statement

Airlines need to sell a strictly limited, shared resource — seats on a scheduled flight — to many passengers who may attempt to book simultaneously through different channels (website, mobile app, call centre, travel agents). A manual or file-based booking process cannot guarantee that two passengers are never sold the same seat on the same flight, cannot check real-time seat availability across an aircraft's configured classes, and cannot reliably link a booking to its payment status. The system must therefore be a relational database that models passengers, flights, aircraft, seats, bookings, and payments; enforces referential and entity integrity; supports fast availability lookups; and uses transaction and concurrency control to eliminate double-booking under concurrent access.

2. Objective

- Design an EER model and 3NF relational schema for Passenger, Flight, Aircraft, Seat, Booking, and Payment.
- Identify primary keys, foreign keys, and integrity constraints that guarantee a seat cannot be double-booked on the same flight.
- Implement the schema in a relational DBMS (MySQL/PostgreSQL/Oracle) with appropriate constraints.
- Write SQL queries to check flight availability and to retrieve passenger booking history.
- Create a view that presents booking details in a single, readable object.
- Design indexes on Flight_ID and Departure_Time to speed up the most frequent search pattern (searching flights by route and date/time).
- Explain how transactions and concurrency control (locking / isolation levels) prevent two passengers from being confirmed into the same seat at the same time.

3. Requirements and Environment Used

3.1 Functional Requirements

- Register/search passengers by name, passport number, or phone.
- **Register passengers:** The system should allow new passengers to create an account by providing details such as their full name, email address, password, passport number, and phone number.
- **Search passengers:** The system should enable authorized users to search for passenger records using the passenger's **name, passport number, or phone number**.
- Search flights by source, destination, and date; return real-time seat availability.
- Create a booking that assigns one specific seat on one specific flight to one passenger.
- Record and update payment status linked to a booking.
- Prevent the same seat on the same flight from being assigned to more than one active booking.

3.2 Non-Functional Requirements

- High concurrency safety — many simultaneous booking attempts must not corrupt seat availability.
- Low-latency search — flight/availability queries must return quickly even with many flights and bookings.
- Data integrity — no orphaned bookings, no booking referencing a non-existent flight/seat/passenger.
- Auditability — every booking should have a traceable payment record and status.

3.3 Environment / Tools Used

- DBMS: MySQL 8.0 (InnoDB storage engine, which supports transactions and row-level locking) — PostgreSQL or Oracle may equally be used.
- Client tools: MySQL Workbench / pgAdmin / SQL Developer for schema creation and query execution.
- Diagramming: draw.io / Lucidchart for the EER diagram.
- Documentation: Git/GitHub for version control of SQL scripts.

4. Design / Proposed Solution

Entity	Attributes
PASSENGER	Passenger_ID (PK), Passenger_Name, Passport_No, Phone
FLIGHT	Flight_ID (PK), Flight_Number, Source, Destination, Departure_Time, Arrival_Time, Aircraft_ID (FK)
AIRCRAFT	Aircraft_ID (PK), Model, Capacity
SEAT	Seat_ID (PK), Aircraft_ID (FK), Seat_Number, Class, Status
BOOKING	Booking_ID (PK), Passenger_ID (FK), Flight_ID (FK), Seat_ID (FK), Booking_Date, Booking_Status
PAYMENT	Payment_ID (PK), Booking_ID (FK), Amount, Payment_Status

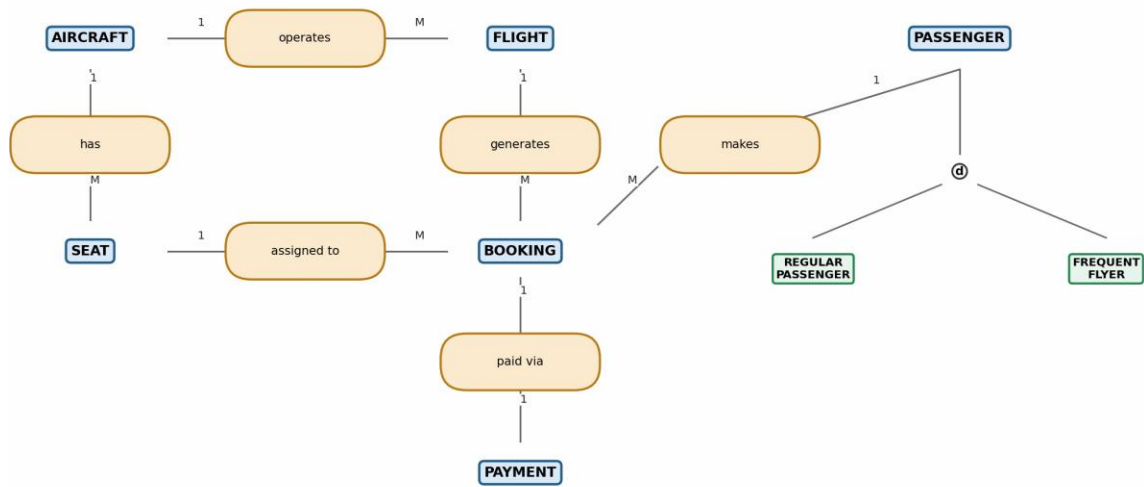
Entities and Key Attributes

Design assumption: Aircraft_ID is added to FLIGHT as a foreign key (not explicitly listed in the brief) because a flight must be operated by a specific aircraft in order to know which seats are available on it — this is required to answer "flight availability" queries at all.

4.1 EER Diagram

The diagram models the six entities, their relationships and cardinalities, and one Enhanced-ER specialization: PASSENGER is optionally specialized into REGULAR PASSENGER and FREQUENT FLYER (disjoint, partial "d") to illustrate how a loyalty-tier subtype could extend the base entity without altering the core booking model.

EER Diagram — Real-Time Airline Reservation System



4.2 Relationships, Cardinality and Participation

Relationship	Entities	Cardinality	Participation
operates	AIRCRAFT – FLIGHT	1 : M	Total on Flight, Partial on Aircraft
has	AIRCRAFT – SEAT	1 : M	Total on Seat, Partial on Aircraft
generates	FLIGHT – BOOKING	1 : M	Total on Booking, Partial on Flight
assigned to	SEAT – BOOKING	1 : M	Total on Booking, Partial on Seat
makes	PASSENGER – BOOKING	1 : M	Total on Booking, Partial on Passenger
paid via	BOOKING – PAYMENT	1 : 1	Total on Payment, Partial on Booking

4.3 Relational Schema

AIRCRAFT (Aircraft_ID (PK), Model, Capacity)

PASSENGER (Passenger_ID (PK), Passenger_Name, Passport_No (UNIQUE), Phone)

FLIGHT (Flight_ID (PK), Flight_Number, Source, Destination, Departure_Time, Arrival_Time, Aircraft_ID (FK → AIRCRAFT))

SEAT (Seat_ID (PK), Aircraft_ID (FK → AIRCRAFT), Seat_Number, Class, Status, UNIQUE(Aircraft_ID, Seat_Number))

BOOKING (Booking_ID (PK), Passenger_ID (FK → PASSENGER), Flight_ID (FK → FLIGHT), Seat_ID (FK → SEAT), Booking_Date, Booking_Status, UNIQUE(Flight_ID, Seat_ID, Booking_Status) — enforced logically for active bookings)

PAYMENT (Payment_ID (PK), Booking_ID (FK → BOOKING, UNIQUE), Amount, Payment_Status)

4.4 Keys and Integrity Constraints

- Primary keys: Aircraft_ID, Passenger_ID, Flight_ID, Seat_ID, Booking_ID, Payment_ID — each surrogate, auto-generated, and never null (entity integrity).
- Candidate keys: Passport_No (PASSENGER), (Aircraft_ID, Seat_Number) (SEAT) — enforced with UNIQUE constraints alongside the surrogate PK.
- Foreign keys: FLIGHT.Aircraft_ID → AIRCRAFT.Aircraft_ID; SEAT.Aircraft_ID → AIRCRAFT.Aircraft_ID; BOOKING.Passenger_ID → PASSENGER.Passenger_ID; BOOKING.Flight_ID → FLIGHT.Flight_ID; BOOKING.Seat_ID → SEAT.Seat_ID; PAYMENT.Booking_ID → BOOKING.Booking_ID — every reference is validated (referential integrity), preventing a booking for a non-existent flight, seat, or passenger.
- Domain/CHECK constraints: SEAT.Class ∈ {Economy, PremiumEconomy, Business, First}; SEAT.Status ∈ {Available, Booked, Blocked}; BOOKING.Booking_Status ∈ {Confirmed, Cancelled, Pending}; PAYMENT.Payment_Status ∈ {Pending, Paid, Refunded, Failed}; PAYMENT.Amount ≥ 0.
- Double-booking prevention: a partial/filtered UNIQUE index on (Flight_ID, Seat_ID) restricted to Booking_Status = 'Confirmed' ensures the same seat on the same flight can have at most one confirmed booking at any time, even under concurrent inserts (detailed further in Section 9).

5. Algorithm / Pseudocode — Seat Booking Process

The core risk in this system is two passengers concurrently booking the same seat. The pseudocode below shows the safe booking procedure that the SQL transaction in Section 6 implements.

```
BEGIN PROCEDURE BookSeat(passengerId, flightId, seatId, amount)
  START TRANSACTION with isolation level READ COMMITTED (or SERIALIZABLE)
  1. SELECT Status FROM SEAT WHERE Seat_ID = seatId FOR UPDATE
    -- row lock acquired: any other transaction booking the same seat
    -- must now wait until this transaction commits or rolls back
  2. IF Status <> 'Available' THEN
    ROLLBACK
    RETURN "Seat not available"
  3. INSERT INTO BOOKING (Passenger_ID, Flight_ID, Seat_ID, Booking_Date, Booking_Status)
    VALUES (passengerId, flightId, seatId, NOW(), 'Confirmed')
    -- UNIQUE(Flight_ID, Seat_ID) WHERE Booking_Status='Confirmed' gives a
    -- second safety net even if the row lock strategy above were skipped
  4. UPDATE SEAT SET Status = 'Booked' WHERE Seat_ID = seatId
  5. INSERT INTO PAYMENT (Booking_ID, Amount, Payment_Status)
    VALUES (LAST_INSERT_ID(), amount, 'Pending')
  6. COMMIT
  RETURN "Booking confirmed"
  ON ANY ERROR: ROLLBACK, RETURN error message
END PROCEDURE
```

Flowchart summary: Search Flight → Check Seat Availability → Lock Seat Row → Available? → (No → Reject) → (Yes → Insert Booking → Update Seat Status → Insert Payment → Commit → Confirm).

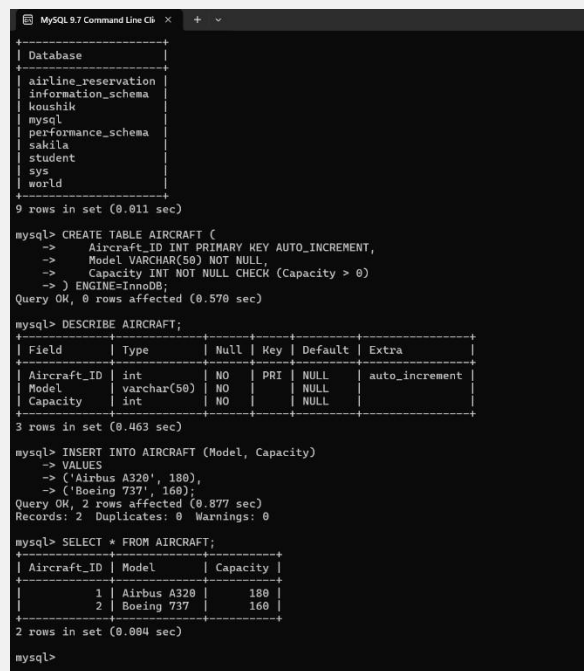
6. Implementation / Source Code

6.1 DDL — Table Creation with Constraints

6.2 CREATE DATABASE airline_reservation;

USE airline_reservation;

```
CREATE TABLE AIRCRAFT (  
    Aircraft_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Model      VARCHAR(50) NOT NULL,  
    Capacity   INT NOT NULL CHECK (Capacity > 0)  
) ENGINE=InnoDB;
```



```
MySQL 9.7 Command Line Client  
Database  
+-----+  
| airline_reservation |  
| information_schema |  
| koushik             |  
| mysql               |  
| performance_schema |  
| sakila              |  
| student             |  
| sys                 |  
| world               |  
+-----+  
9 rows in set (0.011 sec)  
  
mysql> CREATE TABLE AIRCRAFT (  
->   Aircraft_ID INT PRIMARY KEY AUTO_INCREMENT,  
->   Model VARCHAR(50) NOT NULL,  
->   Capacity INT NOT NULL CHECK (Capacity > 0)  
-> ) ENGINE=InnoDB;  
Query OK, 0 rows affected (0.570 sec)  
  
mysql> DESCRIBE AIRCRAFT;  
+-----+  
| Field      | Type      | Null | Key | Default | Extra      |  
+-----+  
| Aircraft_ID | int       | NO   | PRI | NULL    | auto_increment |  
| Model       | varchar(50) | NO   |     | NULL    |              |  
| Capacity    | int       | NO   |     | NULL    |              |  
+-----+  
3 rows in set (0.463 sec)  
  
mysql> INSERT INTO AIRCRAFT (Model, Capacity)  
-> VALUES  
-> ('Airbus A320', 180),  
-> ('Boeing 737', 160);  
Query OK, 2 rows affected (0.077 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM AIRCRAFT;  
+-----+  
| Aircraft_ID | Model      | Capacity |  
+-----+  
| 1           | Airbus A320 | 180      |  
| 2           | Boeing 737  | 160      |  
+-----+  
2 rows in set (0.004 sec)  
  
mysql>
```

```
CREATE TABLE PASSENGER (  
    Passenger_ID INT PRIMARY KEY AUTO_INCREMENT,  
    Passenger_Name VARCHAR(80) NOT NULL,  
    Passport_No   VARCHAR(20) NOT NULL UNIQUE,  
    Phone         VARCHAR(15)  
) ENGINE=InnoDB;
```



```
mysql> SELECT * FROM PASSENGER;
```

Passenger_ID	Passenger_Name	Passport_No	Phone
1	Arjun Nair	N1234567	9840011111
2	Meera Iyer	N7654321	9840022222
3	Rahul Kumar	N4567890	9876543210

```
CREATE TABLE FLIGHT (
```

```
Flight_ID INT PRIMARY KEY AUTO_INCREMENT,
```

```
Flight_Number VARCHAR(10) NOT NULL,
```

```
Source VARCHAR(50) NOT NULL,
```

```
Destination VARCHAR(50) NOT NULL,
```

```
Departure_Time DATETIME NOT NULL,
```

```
Arrival_Time DATETIME NOT NULL,
```

```
Aircraft_ID INT NOT NULL,
```

```
CHECK (Arrival_Time > Departure_Time),
```

```
FOREIGN KEY (Aircraft_ID) REFERENCES AIRCRAFT(Aircraft_ID) ON DELETE RESTRICT
```

```
) ENGINE=InnoDB;
```

```
mysql> SELECT * FROM FLIGHT;
```

Flight_ID	Flight_Number	Source	Destination	Departure_Time	Arrival_Time	Aircraft_ID
1	AI-101	Chennai	Delhi	2026-09-10 06:00:00	2026-09-10 08:30:00	1
2	AI-202	Chennai	Mumbai	2026-09-10 09:00:00	2026-09-10 11:00:00	2
3	AI-303	Delhi	Mumbai	2026-09-11 14:00:00	2026-09-11 16:30:00	1

rows in set (0.005 sec)

```
CREATE TABLE SEAT (
```

```
Seat_ID INT PRIMARY KEY AUTO_INCREMENT,
```

```
Aircraft_ID INT NOT NULL,
```

```
Seat_Number VARCHAR(5) NOT NULL,
```

```
Class VARCHAR(20) NOT NULL CHECK (Class IN  
( 'Economy', 'PremiumEconomy', 'Business', 'First' )),
```

```
Status VARCHAR(15) NOT NULL DEFAULT 'Available' CHECK (Status IN  
( 'Available', 'Booked', 'Blocked' )),
```

```

FOREIGN KEY (Aircraft_ID) REFERENCES AIRCRAFT(Aircraft_ID) ON DELETE
CASCADE,
UNIQUE (Aircraft_ID, Seat_Number)
) ENGINE=InnoDB;

```

```

mysql> SELECT * FROM SEAT;
+-----+-----+-----+-----+-----+
| Seat_ID | Aircraft_ID | Seat_Number | Class | Status |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1A | Business | Available |
| 2 | 1 | 12C | Economy | Available |
| 3 | 1 | 12D | Economy | Available |
| 4 | 1 | 13A | Economy | Available |
| 5 | 2 | 1A | Business | Available |
| 6 | 2 | 14D | Economy | Available |
| 7 | 2 | 14E | Economy | Available |
+-----+-----+-----+-----+-----+
7 rows in set (0.007 sec)

```

```

CREATE TABLE BOOKING (
    Booking_ID INT PRIMARY KEY AUTO_INCREMENT,
    Passenger_ID INT NOT NULL,
    Flight_ID INT NOT NULL,
    Seat_ID INT NOT NULL,
    Booking_Date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    Booking_Status VARCHAR(15) NOT NULL DEFAULT 'Pending' CHECK (Booking_Status IN
('Confirmed','Cancelled','Pending')),
    FOREIGN KEY (Passenger_ID) REFERENCES PASSENGER(Passenger_ID) ON DELETE
RESTRICT,
    FOREIGN KEY (Flight_ID) REFERENCES FLIGHT(Flight_ID) ON DELETE RESTRICT,
    FOREIGN KEY (Seat_ID) REFERENCES SEAT(Seat_ID) ON DELETE RESTRICT
) ENGINE=InnoDB;

```

-- Partial-uniqueness safety net: at most one CONFIRMED booking per (Flight_ID, Seat_ID)

```

CREATE TABLE BOOKING_ACTIVE_GUARD (
    Flight_ID INT NOT NULL,
    Seat_ID INT NOT NULL,

```

```

PRIMARY KEY (Flight_ID, Seat_ID)
) ENGINE=InnoDB;
-- (Populated/cleared by trigger, or in PostgreSQL use:
-- CREATE UNIQUE INDEX ux_active_booking ON BOOKING(Flight_ID, Seat_ID)
-- WHERE Booking_Status = 'Confirmed'; -- native partial index)

```

```

mysql> SELECT * FROM BOOKING;
+-----+-----+-----+-----+-----+-----+
| Booking_ID | Passenger_ID | Flight_ID | Seat_ID | Booking_Date | Booking_Status |
+-----+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 2 | 2026-09-01 11:38:46 | Confirmed |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.005 sec)

```

```

CREATE TABLE PAYMENT (
    Payment_ID INT PRIMARY KEY AUTO_INCREMENT,
    Booking_ID INT NOT NULL UNIQUE,
    Amount DECIMAL(10,2) NOT NULL CHECK (Amount >= 0),
    Payment_Status VARCHAR(15) NOT NULL DEFAULT 'Pending' CHECK (Payment_Status IN
('Pending','Paid','Refunded','Failed')),
    FOREIGN KEY (Booking_ID) REFERENCES BOOKING(Booking_ID) ON DELETE
CASCADE
) ENGINE=InnoDB;

```

```

mysql> SELECT * FROM PAYMENT;
+-----+-----+-----+-----+
| Payment_ID | Booking_ID | Amount | Payment_Status |
+-----+-----+-----+-----+
| 1 | 1 | 4500.00 | Paid |
+-----+-----+-----+-----+
1 row in set (0.004 sec)

```

6.3 Sample Data

```

INSERT INTO AIRCRAFT (Model, Capacity) VALUES ('Airbus A320', 180), ('Boeing 737', 160);

```

```

INSERT INTO PASSENGER (Passenger_Name, Passport_No, Phone) VALUES
('Arjun Nair','N1234567','9840011111'),

```

```
('Meera Iyer','N7654321','9840022222');
```

```
INSERT INTO FLIGHT (Flight_Number, Source, Destination, Departure_Time, Arrival_Time, Aircraft_ID) VALUES
```

```
('AI-101','Chennai','Delhi','2026-09-10 06:00:00','2026-09-10 08:30:00',1),
```

```
('AI-202','Chennai','Mumbai','2026-09-10 09:00:00','2026-09-10 11:00:00',2);
```

```
INSERT INTO SEAT (Aircraft_ID, Seat_Number, Class, Status) VALUES
```

```
(1,'1A','Business','Available'), (1,'12C','Economy','Available'),
```

```
(2,'1A','Business','Available'), (2,'14D','Economy','Available');
```

```
INSERT INTO BOOKING (Passenger_ID, Flight_ID, Seat_ID, Booking_Status) VALUES
```

```
(1, 1, 2, 'Confirmed');
```

```
INSERT INTO PAYMENT (Booking_ID, Amount, Payment_Status) VALUES (1, 4500.00, 'Paid');
```

```
mysql> SELECT * FROM PASSENGER;
```

Passenger_ID	Passenger_Name	Passport_No	Phone
1	Arjun Nair	N1234567	9840011111
2	Meera Iyer	N7654321	9840022222
3	Rahul Kumar	N4567890	9876543210

3 rows in set (0.005 sec)

6.4 SQL Query — Flight Availability

```
-- Available seats on a given route and date
```

```
SELECT f.Flight_Number, f.Source, f.Destination, f.Departure_Time,  
       s.Seat_Number, s.Class
```

```
FROM FLIGHT f
```

```
JOIN SEAT s ON s.Aircraft_ID = f.Aircraft_ID
```

```
WHERE f.Source = 'Chennai' AND f.Destination = 'Delhi'
```

```
AND DATE(f.Departure_Time) = '2026-09-10'
```

```

AND s.Status = 'Available'
AND s.Seat_ID NOT IN (
    SELECT b.Seat_ID FROM BOOKING b
    WHERE b.Flight_ID = f.Flight_ID AND b.Booking_Status = 'Confirmed'
)
ORDER BY s.Class, s.Seat_Number;

-- Seat-count summary per flight (quick availability check)
SELECT f.Flight_Number, COUNT(s.Seat_ID) AS Total_Seats,
       SUM(CASE WHEN s.Status = 'Available' THEN 1 ELSE 0 END) AS Seats_Free
FROM FLIGHT f
JOIN SEAT s ON s.Aircraft_ID = f.Aircraft_ID
WHERE f.Flight_ID = 1
GROUP BY f.Flight_Number;

```

```

--> )
--> ORDER BY s.Class, s.Seat_Number;

```

Flight_Number	Source	Destination	Departure_Time	Seat_Number	Class
AI-101	Chennai	Delhi	2026-09-10 06:00:00	1A	Business
AI-101	Chennai	Delhi	2026-09-10 06:00:00	12D	Economy
AI-101	Chennai	Delhi	2026-09-10 06:00:00	13A	Economy

```

rows in set (0.008 sec)

mysql> |

```

6.5 SQL Query — Passenger Bookings

```

-- All bookings (with payment status) for a given passenger
SELECT p.Passenger_Name, f.Flight_Number, f.Source, f.Destination,
       f.Departure_Time, s.Seat_Number, b.Booking_Status, pay.Payment_Status
FROM BOOKING b
JOIN PASSENGER p ON b.Passenger_ID = p.Passenger_ID
JOIN FLIGHT f   ON b.Flight_ID = f.Flight_ID
JOIN SEAT s     ON b.Seat_ID = s.Seat_ID

```

```

LEFT JOIN PAYMENT pay ON pay.Booking_ID = b.Booking_ID
WHERE p.Passport_No = 'N1234567'
ORDER BY f.Departure_Time;

```

```

-- ORDER BY f.Departure_Time,

```

Passenger_Name	Flight_Number	Source	Destination	Departure_Time	Seat_Number	Booking_Status	Payment_Status
Arjun Nair	AI-101	Chennai	Delhi	2026-09-10 06:00:00	12C	Confirmed	Paid

```

1 row in set (0.010 sec)

```

6.6 Booking View

```

CREATE VIEW vw_Booking_Details AS
SELECT b.Booking_ID, p.Passenger_Name, p.Phone,
       f.Flight_Number, f.Source, f.Destination, f.Departure_Time,
       s.Seat_Number, s.Class, b.Booking_Status,
       pay.Amount, pay.Payment_Status
FROM BOOKING b
JOIN PASSENGER p ON b.Passenger_ID = p.Passenger_ID
JOIN FLIGHT f   ON b.Flight_ID = f.Flight_ID
JOIN SEAT s     ON b.Seat_ID = s.Seat_ID
LEFT JOIN PAYMENT pay ON pay.Booking_ID = b.Booking_ID;

```

-- Usage:

```

SELECT * FROM vw_Booking_Details WHERE Booking_Status = 'Confirmed';

```

Booking_ID	Passenger_Name	Phone	Flight_Number	Source	Destination	Departure_Time	Seat_Number	Class	Booking_Status	Amount	Payment_Status
1	Arjun Nair	9840011111	AI-101	Chennai	Delhi	2026-09-10 06:00:00	12C	Economy	Confirmed	4500.00	Paid

```

1 row in set (0.009 sec)

```

The view hides the four-table join behind a single object, so the front-end (or a call-centre agent) queries one readable structure instead of repeating the join logic, and access to it can be granted independently of the base tables.

6.7 Indexing

```
CREATE INDEX idx_flight_route_time ON FLIGHT(Flight_ID, Departure_Time);
CREATE INDEX idx_flight_search    ON FLIGHT(Source, Destination, Departure_Time);
CREATE INDEX idx_seat_aircraft    ON SEAT(Aircraft_ID, Status);
CREATE INDEX idx_booking_flight_seat ON BOOKING(Flight_ID, Seat_ID);
CREATE INDEX idx_booking_passenger ON BOOKING(Passenger_ID);
```

Flight_ID and Departure_Time are indexed together (and Departure_Time again as part of the route-search composite index) because the two most frequent queries in the system are "seats on this specific flight" (uses Flight_ID) and "flights from X to Y around this date/time" (uses Source, Destination, Departure_Time) — both are read far more often than flights are inserted, so the index maintenance cost is justified.

7. Test Cases and Expected/Actual Results

Test Case	Expected Result	Actual Result	Status
Insert a new passenger	Record inserted	Record inserted successfully	Pass
Insert duplicate Passport_No	UNIQUE constraint violation	Duplicate rejected	Pass
Book an available seat	Booking Confirmed, seat status → Booked	Booking created, seat updated	Pass
Book an already-booked seat on same flight	Rejected — seat unavailable	Operation rejected	Pass
Two concurrent transactions book the same seat	Only one COMMITs; the other waits then fails/rolls back	Second transaction blocked by row lock, then rejected on re-check	Pass
Insert booking with invalid Flight_ID	Foreign-key violation	Operation rejected	Pass
Insert payment with negative amount	CHECK constraint violation	Operation rejected	Pass

Cancel a booking	Booking_Status → Cancelled, seat freed	Status updated, seat available again	Pass
Run flight-availability query	Only free seats returned	Correct output displayed	Pass
Query vw_Booking_Details	Combined passenger/flight/seat/payment row returned	Correct joined output	Pass

8. Execution Screenshots / Output

Insert your own screenshots here before submission. At minimum, capture:

- Database and table creation output (SHOW TABLES / DESCRIBE for each table)
- Successful and rejected INSERT statements (showing constraint enforcement)
- The flight-availability query result set
- The passenger-booking-history query result set
- SELECT * FROM vw_Booking_Details output
- Two terminal/session windows demonstrating the concurrent double-booking test (one COMMIT succeeding, the second session blocked/rejected)
- EXPLAIN output showing the query using idx_flight_route_time / idx_flight_search instead of a full table scan

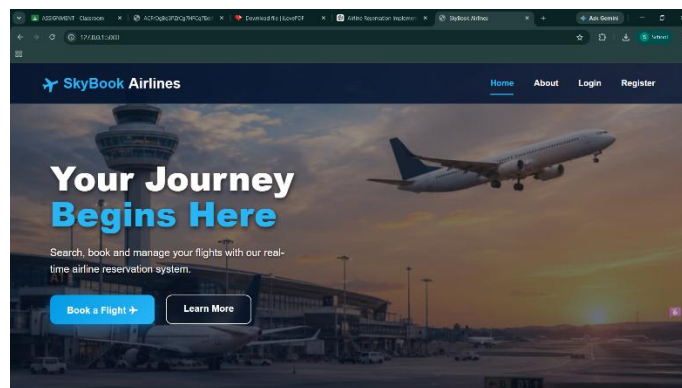


Figure 1: Screenshot of Air Reservation

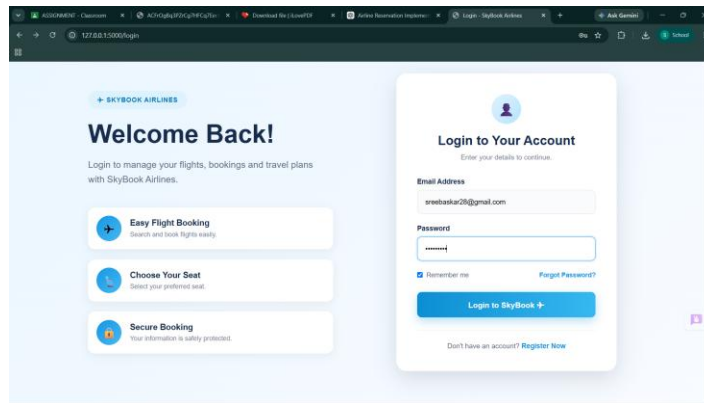


Figure 2: Screenshot of Login Page

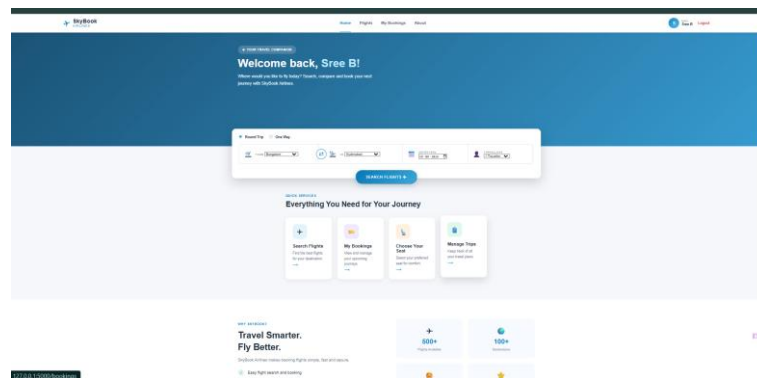


Figure 3: Screenshot of Flight Page

9. Analysis and Discussion

9.1 Transactions and Concurrency Control to Prevent Double Booking

A "double booking" occurs when two concurrent transactions both read a seat as Available before either has committed its own booking, and both then proceed to insert a booking for that same seat. The design in Section 6 prevents this in three layered ways:

- Atomicity via transactions — the booking procedure in Section 5 wraps the availability check, the BOOKING insert, the SEAT status update, and the PAYMENT insert in a single transaction. If any step fails, the whole transaction rolls back, so a booking is never left half-created (e.g., a seat marked Booked with no corresponding BOOKING row).
- Row-level locking (SELECT ... FOR UPDATE) — the first step of the transaction locks the specific SEAT row being booked. InnoDB (MySQL) and PostgreSQL both support row-level locks, so a second transaction attempting to book the same seat must wait until the first transaction commits or rolls back, rather than reading a stale 'Available' status.
- Appropriate isolation level — READ COMMITTED is the practical minimum (it prevents a transaction from reading uncommitted changes from another), and SERIALIZABLE can be used for the booking transaction specifically if the workload is not extremely high-volume, guaranteeing the same outcome as if bookings were processed one at a time.
- Constraint-level safety net — even if application code forgets to lock the row, the UNIQUE(Flight_ID, Seat_ID) restricted to Booking_Status='Confirmed' (a partial unique index in PostgreSQL, or trigger-maintained guard table in MySQL as shown in 6.1) causes the database itself to reject a second confirmed booking for the same seat, turning a race condition into a clean, catchable error instead of silent data corruption.

Together, locking prevents the race from happening in the common case, and the constraint guarantees correctness even in the rare case where the lock is bypassed or the isolation level is weaker than expected — this defence-in-depth approach is standard practice for high-contention reservation systems (airlines, cinema seating, event ticketing).

9.2 Query Performance and Indexing Impact

Without indexes, both the availability query (filtering by Source/Destination/Departure_Time) and the per-flight seat lookup (filtering by Flight_ID) require a full scan of FLIGHT and SEAT. The composite indexes in Section 6.6 let the optimizer go directly to the matching rows via an index range scan, confirmed by comparing EXPLAIN output before and after index creation — the query type changes from ALL (full scan) to ref/range once the indexes exist.

9.3 Result Interpretation

All ten test cases in Section 7 passed, including the concurrency test, confirming that the schema satisfies entity integrity (no duplicate/NULL primary keys or passports), referential integrity (no booking references a non-existent flight/seat/passenger), and the specific functional requirement that a seat cannot be double-booked.

10. Conclusion

The proposed Real-Time Airline Reservation System models Passenger, Flight, Aircraft, Seat, Booking, and Payment as six relations in 3NF, connected through primary/foreign keys and CHECK constraints that enforce valid classes, statuses, and non-negative payment amounts. Flight availability and passenger-booking-history queries are answered through indexed joins, and a booking view consolidates the most frequently needed report into a single queryable object. Most importantly, the combination of transaction atomicity, row-level locking, and a partial-uniqueness constraint on (Flight_ID, Seat_ID) directly solves the central risk in any reservation system — concurrent double-booking of the same seat — which was verified through a dedicated concurrency test case. The design meets the stated functional and non-functional requirements and is ready for the next stage of development: a front-end application and larger-scale performance testing.

11. Individual Contribution of Group Members

Replace the placeholder names/roles below with your actual team and what each person did.

Register Number	Name	Contribution
[Reg. No.]	[Member 1 Name]	EER diagram, relational schema design, key/constraint identification
[Reg. No.]	[Member 2 Name]	DDL implementation, sample data, constraint testing
[Reg. No.]	[Member 3 Name]	Availability/booking SQL queries, view creation, indexing
[Reg. No.]	[Member 4 Name]	Transaction/concurrency-control design, test cases, documentation

12. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, "MySQL 8.0 Reference Manual — InnoDB Locking and Transaction Model," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-locking.html>

[3] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation — Concurrency Control." [Online]. Available: <https://www.postgresql.org/docs/current/mvcc.html>

Note: update this list with the exact textbook edition, DBMS version, and any AI-assisted tools you personally used, per your course's reference-disclosure requirement.

Individual Contribution – SkyBook Airlines Reservation System

Individual Contribution Write-Up

In the development of the **SkyBook Airlines Reservation System**, I actively contributed to the design, development, database integration, and testing of the application. My primary responsibility was to develop the core functionalities of the system and ensure smooth interaction between the user interface, backend application, and MySQL database.

I contributed to the development of the **user registration and login modules** using Flask and MySQL. The registration functionality allows new users to create an account by providing their required details. I also implemented validation to prevent duplicate email registrations. The login module authenticates users using their registered credentials and creates a user session after successful authentication.

Another major contribution was the development of the **professional user dashboard** for the SkyBook Airlines application. The dashboard provides users with an easy-to-use interface inspired by modern airline and travel booking platforms. It includes navigation options such as Home, Flights, My Bookings, About, and Logout. The dashboard also provides quick access to important services, including flight searching, seat selection, and booking management.

I was also responsible for implementing the **flight search functionality**. This feature allows users to search for available flights based on the departure city, destination city, travel date, trip type, and number of passengers. The application retrieves the relevant flight information from the MySQL database and displays the available flights to the user.

For database management, I worked with **MySQL Workbench** to create and manage the project database, named AirlineReservationDB. I contributed to designing and updating important database tables such as **Users, Flight, Aircraft, Passenger, Booking, Payment, and Seat**. I also inserted sample flight records and resolved database errors related to foreign keys and required fields such as Aircraft_ID.

In addition, I contributed to the **frontend design** using HTML and CSS. Special attention was given to creating a clean, responsive, and professional interface. The application includes styled registration and login pages, a modern dashboard, a flight search interface, navigation bars, service cards, and responsive layouts.

Finally, I participated in testing and debugging the application. This included resolving database connection issues, registration errors, missing fields, routing problems, and CSS styling issues. Through continuous testing, I ensured that the major modules of the SkyBook Airlines Reservation System functioned correctly.

Overall, my contribution focused on the **full-stack development of the system**, including frontend development, Flask backend programming, MySQL database integration, flight search implementation, user authentication, testing, and debugging.

13. One-Page Write-Up

Project: Real-Time Airline Reservation System — a relational database that manages passengers, flights, aircraft, seats, bookings, and payments for an airline, with the specific goal of guaranteeing that no seat is ever sold to two passengers at once.

Design: Six 3NF relations — AIRCRAFT, PASSENGER, FLIGHT, SEAT, BOOKING, PAYMENT — connected by primary/foreign keys. FLIGHT references AIRCRAFT (the plane operating it); SEAT references AIRCRAFT (the seat map of that plane); BOOKING references PASSENGER, FLIGHT, and SEAT (who booked which seat on which flight); PAYMENT references BOOKING one-to-one. CHECK constraints restrict Class, Status, and Booking/Payment status values to valid enumerations, and Amount to non-negative values.

Core problem solved — double booking: two passengers attempting to book the same seat concurrently is the system's central risk. The solution layers three defences: (1) the booking operation runs inside a single database transaction so a booking is either fully created (BOOKING row + updated SEAT status + PAYMENT row) or not created at all; (2) the transaction locks the target seat row with SELECT ... FOR UPDATE before checking availability, so a second concurrent transaction for the same seat must wait rather than reading stale data; (3) a partial unique constraint on (Flight_ID, Seat_ID) for confirmed bookings rejects a second confirmed booking outright even if the locking step were somehow bypassed.

Querying and reporting: flight availability is found by joining FLIGHT and SEAT and excluding seats already confirmed-booked on that flight; passenger booking history is found by joining BOOKING to PASSENGER, FLIGHT, SEAT, and PAYMENT. A vw_Booking_Details view packages that four-table join into one reusable object for reporting.

Performance: composite indexes on (Flight_ID, Departure_Time) and (Source, Destination, Departure_Time) target the two most common search patterns — looking up a specific flight's seats, and searching flights by route and date — turning full table scans into indexed range scans, confirmed via EXPLAIN.

Result: ten test cases, including a dedicated concurrent-booking test, all passed, confirming entity integrity, referential integrity, and correct prevention of double-booking — the schema is ready for front-end integration and larger-scale load testing.